

Associate Data Engineer Take-Home Assessment

DHIS2 Health Data Pipeline Challenge

Duration 6–8 hours	Stack PySpark · Python 3.10+	Submission GitHub repo	Domain PSI / DHIS2 / Global Health
------------------------------	--	----------------------------------	--

1. Background

Welcome, and thank you for taking the time to complete this challenge. PSI's DISC team uses DHIS2 as the primary platform for collecting monthly health service data across 32+ countries covering malaria, HIV, TB, reproductive health, and child health programmes.

Raw data arrives from country DHIS2 instances as JSON exports containing cryptic 11-character UUIDs that must be resolved against metadata to become meaningful. The org unit hierarchy — facility → district → region → national — is stored as a self-referencing tree. Data quality issues including late reporting, zero-vs-NULL ambiguity, and ghost UUIDs are a fact of life in every country export.

Your task is to build the ELT pipeline that transforms four raw DHIS2 JSON files into a clean dimensional warehouse, apply program-specific business logic, and produce analytics-ready outputs a programme manager can query directly. This mirrors the actual data flow: country DHIS2 → PSI data lake → analytics layer.

This task is designed to take 6–8 hours. We value sound engineering judgement and clear thinking over completeness. If you hit a genuine blocker, document it in your README and move on.

2. Getting Started

Generate the dataset

A data-generation script is provided alongside this document. Run it once to produce your four input JSON files, then work only from those outputs.

```
pip install faker numpy
python generate_data.py                # default: 5 countries, 12 periods
python generate_data.py --countries 8 --periods 18 --seed 42
```

Run your pipeline

```
pip install pyspark==3.4.*
python pipeline.py --data-dir ./data --output-dir ./output
```

3. Dataset: DHIS2 Export Files

The generator produces four JSON files that mirror the real DHIS2 Web API response structure. Every value in `data_values.json` is a string — DHIS2 always serialises this way, regardless of the underlying data type.

`metadata.json`

Contains `dataElements[]` and `categoryOptionCombos[]`. Each data element carries a `valueType` (INTEGER_ZERO_OR_POSITIVE, PERCENTAGE, NUMBER, BOOLEAN) that governs how its string value should be cast in your pipeline.

Field	Type	Notes
<code>id</code>	STRING (UID)	11-char alphanumeric DHIS2 UID — all starts with a letter
<code>name</code>	STRING	Human-readable indicator name
<code>valueType</code>	STRING	INTEGER_ZERO_OR_POSITIVE NUMBER PERCENTAGE BOOLEAN
<code>domainType</code>	STRING	AGGREGATE (all records in this export)
<code>categoryCombo.id</code>	STRING (UID)	Links to the disaggregation combo for this element

`org_units.json`

Contains `organisationUnits[]` with a 4-level hierarchy. The `path` field encodes the full ancestry as `/L1_uid/L2_uid/L3_uid/L4_uid`. Only level-4 facilities submit data values.

Field	Type	Notes
<code>id</code>	STRING (UID)	Unique org unit identifier
<code>name</code>	STRING	Org unit display name
<code>level</code>	INTEGER	1 = National, 2 = Region, 3 = District, 4 = Facility
<code>path</code>	STRING	<code>/nat_uid/reg_uid/dist_uid/fac_uid</code> — split on / for hierarchy
<code>parent.id</code>	STRING (UID)	Direct parent; NULL for level-1 (national)
<code>groups[].name</code>	STRING array	Facility type (Health Centre, Hospital, Dispensary, etc.)

`programs.json`

One program per country-health-area combination. Each program lists the data element UIDs it expects facilities to report. Use this to compute your expected-indicators denominator for completeness scoring.

Field	Type	Notes
<code>id</code>	STRING	Program identifier
<code>healthArea</code>	STRING	Malaria HIV Tuberculosis Reproductive Health Child Health
<code>country</code>	STRING	PSI country name
<code>reportingFrequency</code>	STRING	MONTHLY for all programmes in this export

Field	Type	Notes
dataElements	ARRAY<STRING>	UID list of indicators this programme expects

data_values.json

The main ELT input. A dataValueSets-format export: one top-level dataValues[] array containing one object per reported indicator per facility per period. Period format is yyyyMM (e.g. 202401 = January 2024).

Field	Type	Notes
dataElement	STRING (UID)	References metadata.json dataElements[].id
period	STRING	yyyyMM format — e.g. 202401. No date separators.
orgUnit	STRING (UID)	References org_units.json id
categoryOptionCombo	STRING (UID)	Disaggregation combo — may be orphaned
value	STRING	Always a string; cast per valueType. May be null or '0'.
storedBy	STRING	Username of the reporter — casing inconsistent
lastUpdated	STRING	ISO 8601 timestamp. >60d after period end = late-reported.

4. Injected Data Quality Issues

Every issue listed below is deliberately present in the generated data. Your pipeline must handle each one explicitly — not silently. Candidates who treat these as real programme reporting problems rather than generic data cleaning steps will stand out.

Quality issues your pipeline must address

- ~8% exact duplicate dataValue rows
- ~5% dataValues referencing dataElement UIDs absent from metadata.json (unresolvable)
- ~4% dataValues referencing ghost orgUnit UIDs not in org_units.json (orphaned)
- ~3% dataValues with orphaned categoryOptionCombo UIDs
- ~12% late-reported rows: lastUpdated more than 60 days after the period end date
- ~6% explicit zero values (value = "0") — meaningfully different from NULL in health reporting
- ~3% null raw_value — facility submitted a row but provided no value
- ~2% near-duplicate rows: same composite key, corrected value, later lastUpdated timestamp
- ~8% facilities that silently skip one or more reporting periods entirely

Key distinction: a facility reporting 0 clients served is fundamentally different from one that did not report at all. Your pipeline must preserve this distinction — never collapse 0 and NULL.

5. Tasks

Complete all eight tasks in order. Each task's output is the input to the next. A pipeline.py skeleton is provided with the correct SparkSession setup, function signatures, and dependency chain — your job is to fill in the implementation.

#	Task	What to build	Key skills
01	JSON ingestion & schema flattening	Parse nested DHIS2 JSON with explicit schemas; flatten dataValues[]; quarantine malformed records.	SparkSession · from_json · schema
02	Metadata UID resolution	Replace all UID references with human-readable names using broadcast-optimised joins; log unresolvable UIDs.	broadcast · left_anti join
03	Org unit hierarchy resolution	Self-join org units to resolve facility → district → region → country; add columns to every data value row.	self-join · level traversal
04	Data quality & late-reporting flags	Cast raw strings per valueType; flag 0 vs NULL; detect late-reported rows (>60 d); compute completeness score.	when/otherwise · datediff
05	Dimensional model build	Design and populate star schema (fact + 4 dims); write Parquet partitioned by health_area / year_month.	star schema · partitionBy
06	Program analytics & window functions	MoM % change, 3-month rolling avg, country reporting rate, top-5 underreporting facilities per health area.	Window · partitionBy · rank
07	Cross-country aggregation	Global volumes by quarter, completeness comparison, data element coverage matrix, low-completeness flags.	groupBy · pivot · agg
08	Pipeline orchestration	Entry point running Tasks 1–7 in order; stage-level logging; DQ check exit codes; single-command execution.	logging · sys.exit

Task 01 — JSON ingestion & schema flattening

Load all four DHIS2 JSON files using explicit schemas defined in code. Never use inferSchema=True. Explode the nested dataValues[] array into a flat DataFrame (one row per data value). Collect rows that violate the schema into a quarantine output — log them and write them separately rather than dropping silently.

Task 02 — Metadata UID resolution

Join data values to the metadata DataFrames to replace all UID references with human-readable names. Apply a broadcast hint to the metadata lookups (they are small). Use an anti-join to isolate rows whose UIDs have no matching metadata entry. These unresolvable rows must be counted, logged, and written to a separate output — not silently discarded.

Task 03 — Org unit hierarchy resolution

Self-join the org unit table to resolve each facility's full ancestry: facility → district → region → country. Add country_name, region_name, district_name, facility_name, and facility_level as columns on every data value row. Do not hardcode any UIDs or assume a fixed number of levels — traverse the path column dynamically.

Task 04 — Data quality & late-reporting flags

Cast raw_value strings to the correct Python/Spark type based on each data element's valueType from metadata. Add is_late_reported (lastUpdated > 60 days after period end), is_explicit_zero, and

is_missing_value boolean columns. Compute a completeness score per facility per period: count of reported indicators divided by the expected indicator count from programs.json.

Task 05 — Dimensional model build

Design and populate a star schema: fact_service_delivery (one row per data element × period × facility) and four dimension tables — dim_data_element, dim_org_unit, dim_period, dim_program. Write fact_service_delivery as Parquet partitioned by health_area and year_month. All writes must use mode('overwrite') to be idempotent.

Task 06 — Program analytics & window functions

Using PySpark window functions (not groupBy alone), compute: (1) month-over-month percentage change per indicator per district, (2) 3-month rolling average per facility per indicator, (3) country-level reporting rate (percentage of expected facilities that submitted any data per period), (4) top-5 underreporting facilities per health area ranked by periods with zero data.

Task 07 — Cross-country aggregation

Aggregate across all countries to produce: total service volumes by health area by quarter, a cross-country completeness comparison table, a data element coverage matrix (which indicators each country reports, using pivot), and a list of countries consistently below 80% completeness for 3 or more consecutive periods.

Task 08 — Pipeline orchestration

Write a pipeline.py entry point that runs Tasks 1–7 in dependency order. Log row counts and DQ check results at the completion of each stage. Exit with a non-zero code if any critical DQ check fails (e.g. quarantine rate > 10%, 0 rows in fact table). No Airflow required — clean Python orchestration with the logging module is sufficient.

6. Bonus Challenges

These are optional. Attempting the core eight tasks well is worth more than rushing to complete all bonuses. That said, strong candidates often find one or two that interest them.

- **B1 — Data contract validation:** Define a YAML schema contract for fact_service_delivery specifying column types, nullable constraints, and expected value ranges. Write a Python function that validates the Parquet output against this contract before the final write and raises an exception if checks fail.
- **B2 — Incremental load logic:** Modify the pipeline to support incremental ingestion — only process periods not already present in the Parquet output. Use the existing partition structure to detect what has already been loaded.
- **B3 — Anomaly detection:** Flag facilities where any indicator value is more than 3 standard deviations from that facility's own 12-month rolling mean. Output these as a separate anomalies.csv for programme review.
- **B4 — Metadata drift detection:** Compare the current metadata.json to a reference snapshot. Report any data elements that have been added, removed, or renamed between exports. This simulates handling DHIS2 metadata changes between monthly country pulls.

7. Evaluation Rubric

Total available: 100 points + up to 20 bonus points. We use this rubric as a guide, not a checklist — a submission that nails six tasks thoughtfully will score higher than one that attempts all eight superficially.

Area	What we look at	Points
JSON parsing & schema handling	Robust nested JSON flattening. Explicit schemas only. Quarantine pattern for bad records. No inferSchema.	20
UID resolution & joins	All UIDs resolved. Broadcast hint applied. Unresolvable UIDs logged, not silently dropped.	15
Org unit hierarchy	Correct self-join traversal. No hardcoded level UIDs. Handles variable depth gracefully.	15
Dimensional model design	Clean star schema. Correct partitioning strategy. Fact/dim separation is clear and consistent.	15
Analytics & window functions	MoM %, rolling avg, completeness all correct. Native Spark functions. No unnecessary Python UDFs.	15
DQ & observability	Late-report flagging correct. 0 vs NULL handled explicitly. Completeness score computed. Stage logging.	10
Code quality & README	Modular, readable, documented. Runs end-to-end with one command. Design decisions explained.	10
Bonus (each)	Quality of any completed bonus challenge.	+5

What separates a strong submission

- Treating UIDs as opaque keys that require careful resolution, not just column names to rename.
- Handling 0 vs NULL as a real programme reporting distinction, not just a data cleaning step.
- Modelling the org unit hierarchy dynamically — no hardcoded UIDs, no assumed depth.
- Using native Spark functions (F.when, F.datediff, F.broadcast) instead of Python UDFs for simple operations.
- A README that explains design decisions, not just setup instructions.

8. Deliverables

Submit a link to a public (or shared) GitHub repository containing all of the following. The pipeline must run end-to-end with a single command from the project root.

1. **README.md** — setup instructions, how to run, star schema diagram (ASCII or image), design decisions, assumptions, known limitations.
2. **pipeline.py** — the orchestrated entry point. All 8 tasks run in order with stage-level logging. Must execute with: `python pipeline.py --data-dir ./data --output-dir ./output`
3. **models/** — PySpark transformation modules (one per task or logical group). Clean separation of concerns.
4. **output/** — sample Parquet and CSV outputs committed, or clear README instructions to reproduce them.
5. **requirements.txt** — all dependencies pinned. No Airflow, Databricks SDK, or cloud-provider packages required.
6. **tests/** — (Bonus B1) pytest test files if you completed the data contract bonus.

Environment note

We will run your pipeline locally with PySpark 3.4 on Python 3.10. Your SparkSession must be created inside the script — do not assume a running cluster, Databricks environment, or cloud storage. Use local[*] as the master.

9. Submission

When you are ready, send your GitHub repository link to your recruiting contact with the subject line:

[Associate DE] DHIS2 Pipeline Submission – [Your Full Name]

Please include in the email body:

- A one or two sentence summary of the overall approach you took.
- Any tasks you were unable to complete and why.
- Approximate time spent.
- Any assumptions you made that are not documented in your README.

Good luck!

We read every submission carefully. The DHIS2 domain is niche — don't worry if the health data terminology is unfamiliar. We care far more about how you reason through ambiguity and structure your pipeline than whether you know what an ACT dose is.